Inf Syst E-Bus Manage (2016) 14:533–576
DOI 10.1007/s10257-015-0294-3

ORIGINAL ARTICLE

Knowledge base support for dynamic information system management

Adriana Caione[1] · Anna Lisa Guido[1] · Angelo Martella[1] · Roberto Paiano[1] · Andrea Pandurino[1]

Received: 20 March 2015 / Revised: 14 September 2015 / Accepted: 18 September 2015 /
Published online: 24 September 2015

Abstract Enterprise activities are governed by regulations and laws that are multiple, heterogeneous and not always easy to understand. The arising and/or the modification of these regulations and laws can cause a significant impact in the business context, especially in terms of enterprise information systems adaptation. Currently, there are many methodological and technological tools that facilitate the application of regulations and procedures, but they are not integrated enough to ensure a complete problem management. Therefore, they are not sufficient to support organizations and companies in the management of their business processes. In this paper we propose a methodological and technological solution, able to model, manage, execute and monitor business processes of complex domains. The system allows both the design of an information system and its prototyping as a web application, by the extension of an appropriately selected Business Process Management suite. During both the design and the usage phases of the prototyped information system, it is possible to interface with a knowledge base that contains information about regulations and aspects that characterize the enterprise (organizational chart, tasks, etc.).

✉ Adriana Caione
adriana.caione@unisalento.it

Anna Lisa Guido
annalisa.guido@unisalento.it

Angelo Martella
angelo.martella@unisalento.it

Roberto Paiano
roberto.paiano@unisalento.it

Andrea Pandurino
andrea.pandurino@unisalento.it

[1] Department of Engineering for Innovation, University of Salento, 73100 Lecce, Italy

Keywords Enterprise information systems · Knowledge base · Business process · Model driven engineering · User experience · Application prototyping

1 Introduction

From the 1970s to the early 1990s, most of the attention on information systems was oriented towards data. In particular the focus was on both the information storage and retrieval. During the 1990s the trend has shifted the focus to processes. As a result, information systems began to be supported by the business processes modelling and execution; different approaches have been developed such as workflow management systems. Such approaches were applied for the design and implementation of enterprise application-integration tools (Dumas et al. 2005).

In general, business processes are strongly influenced by regulatory issues: who is involved in the business processes execution must perform his/her activity following specific regulations and laws.

As regards, in (Monakova and Leymann 2013) the authors say, 'Business processes are obliged to follow numerous constraints, such as compliance regulations, service level agreements, security regulations and budget constraints. To be able to understand relations between different constraint types and their impact on the business, a common constraint specification framework is required.

Restricting the scope to the Health, Safety and Environmental Protection (HSEP) context, regulations may be issued by different authorities, inside and outside the companies, and are essentially characterized by the following properties: they are manifold, complex and, especially, subject to change. Regulations and laws cannot be ignored in order to avoid economic sanctions and especially to not expose the health and safety of workers and workplaces to risks.

At the same time, it is important to note what is asserted in (Mu et al. 2015): 'Business process management principles are commonly used to improve processes within an organization. But they can equally be applied to supporting the design of an Information System'.

Therefore, a business process-oriented information system which is not regulations-compliant and/or not particularly dynamic to adapt with respect to the regulations changes, may reduce the value added to the organizations.

In general, as suggested in (Millet et al. 2009), 'The alignment of business processes and information systems is a critical factor for both Business Process Management (BPM) and enterprise resource planning (ERP) system efficiency. Analysing existing approaches of alignment shows the need for an independent reference model to support the mapping between organisational and informational views'.

In such a context, arises the research project called HSEPGEST-Management of Health, Safety, and Environmental Protection in enterprise processes, which is a project financed by European and National funds, focusing on the design and development of an integrated management system. During the whole modelling process implemented by such system, the designer can refer to the concepts present

in the knowledge bases, including information concerning regulations, enterprise processes and business processes.

In the present paper the following issues are faced:

- The organizations and, accordingly, the related business processes must comply with clear and complex regulatory frameworks, often deriving from different authorities;
- Because the regulations are subject to numerous changes, both enterprise information systems and process management systems must dynamically adapt to such changes;
- Considering that specific knowledge about both regulations and operational context is the prerogative of only a few domain experts, there is the necessity to transfer some know-how to the business process experts, in order to reduce the knowledge gap;
- The know-how related to both regulations and enterprise structure should characterize the User Experience, as well as business processes, of an enterprise information system.

The solution we propose intends to overcome some typical gaps of the existing business process management systems, implementing a specific integrated architecture. In effect, such architecture is closely connected to the information domain and is developed by business process modeling. Furthermore, the architecture supports process instance execution using simple web interfaces.

For this reason, all the domain information is organized into appropriate knowledge bases, depending on the corresponding fields (i.e. related to regulations, enterprise, business processes). In this regard, we can mention the ARISTOTELE project (Nostro et al. 2013) whose purpose is to sustain enterprises during knowledge management activities. In effect, the models defined in ARISTOTELE project can:

- Efficaciously represent the organization;
- Support the knowledge worker activities within knowledge-intensive organizations;
- Be compliant with the reference methodologies for the domain knowledge management.

Considering such mentioned problems, the purpose of the present paper is the resolution of the above issues, by designing and implementing a framework which consists of the following components:

- A dynamic tool, able to support the business analyst during the modelling and the subsequent execution of business processes. All that has to be done in a way that is compliant with both the enterprise structure and the regulations and laws framework;
- A methodological tool, able to support the user interface designer during the modelling and the generation of the corresponding User Experience prototype.

The dynamic feature of the system is guaranteed by the support provided by some knowledge bases, along with the possibility to model and to implement business processes, without excessive programming efforts. These processes are fully and in a transparent way integrated with the enterprise information system, so the user can directly manage them.

The interaction with the knowledge bases allows sharing the domain know-how among the business analysts. In that way, analysts are able to semantically characterize business processes, and to adapt them to regulations and laws as well. Moreover, the knowledge bases concepts related to both organization and regulator domains, makes changes to the information system more flexible, along with the business process management system.

In the following, we present a framework which consists of an integrated architecture for business process management, along with a tool based on a specific methodology, the designer uses to model the user interfaces of a web application prototype. The emphasis is on the use of knowledge bases in the different stages of process design, interface modelling and process instance execution. The business process management system that we propose is independent from the application domain and, therefore, it can be used in different application scenarios.

Section 2 describes the related works regarding the tools and the methodologies we have referred to for modeling and implementing the framework, along with a background on the integrated project management systems.

While Sect. 3 focuses on the modelling aspects of the concepts related to regulations and laws, Sect. 4 introduces the modelling and the management of the knowledge for the domain of interest. Section 5 presents the proposed system architecture, along with all its modules. Section 6 focuses on the integration of business processes in the User Experience design of the corresponding prototype. Section 7 shows a case study concerning a process of health monitoring, in order to clarify how the architecture works. Finally, in Sect. 8 we present some conclusions and the main challenges ahead.

2 Related works

In (Beckmerhagen et al. 2003), integration is defined as 'a process of putting together different function-specific management systems into a single and more effective integrated management system (IMS)'.

In general, an integrated management system is a set of regulations and procedures, based on international standards that an enterprise can implement in order to achieve goals such as customer satisfaction, performance improvement, etc. The idea behind an IMS is that the enterprise must be able to manage its activities referring to a single information system.

On the other hand, as also asserted by (Pojasek 2006): 'There has long been a search for the "holy grail" of management systems: the fully integrated business management system (or "umbrella system") that can link all the management standards and systems used by the organization'.

For this purpose, in (Bernardo et al. 2009) the authors identify three distinct levels of integration of management systems among companies: strategic level, tactical level and operational level.

In particular 'the strategic level concerns integrated planning and resources deployment; the tactical level concerns the design of IMS and monitoring the execution of IMS; and the operational level concerns the execution of activities in an integrated fashion' (Asif et al. 2010).

Most of the integration strategies in literature use one, or a combination, of the following concepts: the Matrix Model, 'a system of systems', process-based approach, risk-based approach, Total Quality Management (TQM) and Business Excellence (BE) models.

The strategy based on processes considers that the same are described by flowcharts, using specific notation. These descriptions keep track of all aspects of the process, in addition to the requirements of each standard.

This approach is strongly supported by Tranmer (1996), according to which 'the common element of all management systems is a very simple one: process'. In that regard, however, a potential risk is associated with the possibility to 'not consider normative and strategic issues in an adequate way' (Seghezzi 1997).

In this respect, we quote the methodology proposed in (Sadiq and Governatori 2015) that aims to align both control and business objectives; in particular control objectives stem from regulations and legislation. The authors use the term 'process model enrichment as the ability to enhance enterprise models (business processes) with compliance requirements. This can be provided as process annotation'. The methodology improves the understanding of both process owners and compliance officers. Each task in a process model can have associated semantic annotations derived from Formal Contract Logic (FCL) expressions of compliance requirements. All that entails a complete redesign of the process, along with the generation of an execution trace of the same process. Finally, 'A process is compliant if and only if all traces are compliant (all obligations have been fulfilled or if violated they have been compensated). A process is weakly compliant if there is at least one trace that is compliant'.

This paper continues with a description of the key concepts that will be referenced later in the paper.

2.1 Business process management tools

A workflow system provides the ability to encode and to automate the organization of the work. Among several definitions of the workflow term, we quote the most authoritative from the Workflow Management Coalition (www.wfmc.org), where a workflow is defined as 'the automation of a business process or of a part thereof, during which documents, information or tasks are passed from one participant to another in order to be processed in compliance with a set of procedural regulations'.

During the preliminary phases of this work, we have carried out a review of open source technologies and workflow engine frameworks.

Among the open source workflow management solutions, we have identified the following:

- Intalio BPMS (www.intalio.com)
- BonitaSoft (http://it.bonitasoft.com)
- jBPM (www.jbpm.org)
- Activiti BPMN 2.0 (www.activiti.org)
- Camunda (www.camunda.com)
- YAWL (www.yawlfoundation.org)

The following subsections provide some details about all the above workflow engine solutions.

2.1.1 Intalio BPMS

Intalio represents a solution which is independent from any other proprietary technology and which provides all the components useful for the design, deployment and management of complex business processes, including the activities of business monitoring, regulation management, document management and systems integration. Intalio is intended as a reference solution for business users who need a process management platform and also provides a tool for web application management. However, in order to obtain an application which is process-compliant, it is essential to obtain the support of technical experts, able to both customize the process and configure the whole system.

2.1.2 BonitaSoft

Bonita BPM represents a business process management solution, which has the advantage of combining three systems in one: an innovative environment for process design, a powerful engine for both workflow and business process management and a revolutionary user interface. Bonita is characterized by the rapidity whereby it is possible to create applications based on processes and it is particularly addressed to business users that aim to independently manage their processes, without the necessity to refer to expert support.

2.1.3 jBPM

jBPM is a flexible Business Process Management system developed by JBoss, which is lightweight, completely open-source (licensed under Apache) and Java-based. The version 6 jBPM suite allows modelling business processes using BPMN and BPMN 2.0 specifications, along with executing and monitoring the same processes during their life cycle. jBPM focuses on executable business processes, which represent particular processes characterized by sufficient detail information, in order to be executed by a BPM engine. It is important to note that business processes need support during their entire life cycle: creation, implementation, processes and tasks list management, dashboard and report activities.

2.1.4 Activiti BPMN 2.0

Activiti BPMN 2.0 represents the business process management engine on which is currently based the Alfresco platform for business document management. It is an open source solution for Business Process Management, Apache-licensed and designed to implement the version 2.0 BPMN standard by OMG (Object Management Group). Activiti represents a lightweight solution and is particularly designed for Java developers and not for business users.

2.1.5 Camunda

Camunda BPM is a Java-based framework for process automation. It is built around the process engine component, responsible for the execution of BPMN 2.0 processes and workflows. The service-oriented API allows Java applications to directly interact with the process engine.

2.1.6 YAWL

YAWL stands for Yet Another Workflow Language and it is a BPM/workflow system that makes use of a language based on workflow patterns. The core component concept in YAWL is derived from Petri Nets: the workflow represents a set of tasks, flows and conditions between them (Curcin et al. 2007). Among its features, YAWL is characterized by:

- A model for data definition and manipulation based on XML Schema, XPath (www.w3.org/TR/xpath/) and XQuery (www.w3.org/TR/xquery/);
- XML-based interfaces for workflow instance monitoring and execution;
- XML-based plugin interfaces that enable connection with third-party web services;
- Automated form generation from XML schema.

Unlike the above mentioned solutions, YAWL makes use of a notation to represent business processes that is not BPMN standards compliant. Unfortunately such requirement represents a key aspect for the HSEPGEST project, and that's the main reason the YAWL solution has been discarded.

In order to identify the most appropriate tool, the analysis we have conducted focused on the following aspects:

- Existence of an editor that is compliant with the standard notation of process representation (BPMN);
- Existence of APIs to implement the integration with other systems;
- Existence of support for fast prototyping;
- Existence of a console for process monitoring;
- Possibility to customize the application related to the workflow engine console;
- Possibility to customize the solution source code.

Table 1 Comparative analysis between open source workflow engines

	Intalio	Bonita	jBPM	Activiti	Camunda
Editor	Oriented to business users with the support of IT experts	Oriented to business users	Oriented to business users with or without the support of IT experts	Oriented to IT experts	Oriented to IT experts
API	The engine can not be changed	IT experts	Both IT experts and less expert users	IT experts	IT experts
BPMN2.0	Compliant	Compliant	Compliant	Compliant	Compliant
Fast prototyping	Available but it is necessary the support of experts	Available	Available but his necessary the support of experts	Available but it is necessary the support of experts	Available but his necessary the support of experts
Update of the console application	Allowed	Allowed	Not allowed	Not allowed	Not allowed
Business process monitoring	Oriented to business users	Oriented to business users	Oriented to business users and to developers	Oriented to developers	Oriented to developers
Update of the source code	–	–	Availability of source code. jBEM community support	Source code not well documented	Availability of source code. Camunda community support

The following table (Table 1) summarizes the comparative analysis carried out with respect to all the considered solutions.

The comparative analysis between the solutions guided us in choosing jBPM. In effect, it provides a complete and stable tool suite able to model and to execute processes, along with the possibility to monitor sessions, tasks and instances. The jBPM workspace has the main characteristic of being web accessible and is able to interface with the Eclipse-based editor, using a repository. jBPM makes available its own Eclipse plugin that is particularly oriented to developers. The repository allows importing jBPM projects, created using the Eclipse-based editor, into the console. Particularly relevant to our choice are the following aspects:

- Flexibility: jBPM offers an high level of flexibility in terms of customization of its work environments;
- Open source project: jBPM, as an open source project, makes possible to download and to extend its source code. This characteristic represents the most important aspect for the purpose of our work.

2.2 Semantic business process and BPM system integration

One of the most significant attempts of semantic technologies application to business processes is the SUPER project (www.ip-super.org). In particular, the SUPER project focuses on business process management, and introduces the concept of Semantic Business Process, which allows:

- Modelling both business processes and domain information;
- Making the process management accessible to domain experts;
- Modelling and managing business processes in a more efficient and effective way;
- Making inferences and queries on business processes.

The framework called *SUPER semantic BPM platform—SSBPMP* (El Kharbili and Pulvermueller 2009), represents another possible implementation.

SSBPMP proposes a policy-based framework that is part of the business process compliance management, and whose architecture is achieved through the extension of what is already provided by the SUPER research project on semantic business process management (SBPM).

Similar to the solution we propose, *SSBPMP* aims to define a comprehensive and integrated architecture for the compliance semantic modelling and execution in the BPM context.

The fundamental concept underlying *SSBPMP* is the policy, and in particular its semantic representation, which is used by the enterprise process designers to model the business process compliance with laws and regulations.

SSBPMP can be considered as an extension of the ontology stack provided by the SUPER BPM, using a specific ontology related to policies and regulations modelling, which is able to support the compliance management within BPM in an integrated way.

Regarding the generation of semantic representation of a policy, the approach proposed by *SSBPMP* uses as input the corresponding documents related to the same policy, expressed in natural language. In effect, the framework automatically processes such documents in order to translate them into a specific structured language, which is especially suited to the policy semantic representation.

Such policy semantic representations are then used as part of the business process modelling, by defining some anchor points associated with the constituent elements of the process itself, and that refer to a policy assertion.

Actually, besides representing an incomplete solution from the design point of view, *SSBPMP* is absolutely devoid of a reference implementation.

Finally, compared to the solution we propose, *SSBPMP* has the following characteristics:

- Appears limited compared to the use of semantic concepts as part of the process modelling, referring solely to the compliance management;
- Does not provide a specific User Experience—UX—modelling environment for the corresponding prototype generation, through which it is possible to integrate,

in a transparent way for the user, semantic concepts retrieved from the knowledge base, and references to workflow engine business processes.

2.3 User experience design: interactive dialogue model methodology

The Interactive Dialogue Model—IDM—methodology (Bolchini and Paolini 2004) is based on the concept that the use of interactive applications can be assimilated to a dialogue between user and application. IDM, therefore, aims to improve and optimize the interaction paradigms between software product and user.

The main strengths of IDM are the simplicity of the notation and the ability to focus on design details.

The IDM methodology is developed in three phases:

- C-IDM-Conceptual IDM: represents the modelling phase of the contents to be presented to the user, along with their respective relationships. In effect, the C-IDM design phase provides a rather coarse notation, which avoids any possible reference to graphs and technological aspects of the software product to be implemented;
- L-IDM-Logical IDM: is the design phase in which the designer tries to better detail the information content provided by C-IDM, but also to introduce strategies and navigation methods for the contents;
- Page Design: represents the design phase in which each application page is detected and characterized, also from the look and feel point of view.

Finally, Rich-IDM (Pandurino et al. 2010) represents a methodology for the design of Rich Internet Application—RIA—user interfaces, which is obtained by extension of the concepts provided by the IDM methodology. In particular, Rich-IDM extends the IDM methodology to RIA applications, trying to increase the detail level provided by the logical model, in order to obtain advanced user interfaces that minimize the effects due to sense of loss, incorrect mental model of the dialogue and user distraction.

2.4 Model-driven architecture and model-driven engineering

In general, every prototyping process represents a great opportunity for companies, even if it could be highly risky and very expensive in terms of costs.

The scientific literature has identified various methodologies that try to dominate this complexity, in an attempt to standardize the prototyping process using automatic support tools.

However, despite several lines of research, at present there is no particular methodology that is able to minimize the costs and risks of failure, related to the prototyping problem.

Particular progress has been observed in the field of specific methodological prototyping *framework*, which can be configured and used in various scenarios.

In this regard, organizations such as OMG—Object Management Group—are pushing for the adoption of a Model-Driven approach (www.omg.org/mda), able to make systematic and automatic the prototyping process.

Model-Driven Engineering—MDE—and *Model-Driven Architecture—MDA*—terms respectively refer to the development of systems and to the corresponding software architecture.

As reported in (Kent 2002) "The OMG's Model Driven Architecture (MDA) strategy envisages a world where models play a more direct role in software production, being amenable to manipulation and transformation by machine. Model Driven Engineering (MDE) is wider in scope than MDA. MDE combines process and analysis with architecture".

In particular, implementing the prototyping process using the Model-Driven approach means defining meta-models and models capable of characterizing the information flow provided by the process itself, even with respect to each intermediate result. In effect, compared to the reference methodology for prototyping process, the Model-Driven approach provides that:

- One or more models must correspond to each phase;
- A target model represents the result obtained by applying a specific transformation on one or more source models.

2.5 Conclusions

In this section we have presented some technologies and methodologies we have referred to for modeling and implementing the framework, along with a background on the integrated project management systems. Following we briefly report how we have used such technologies and methodologies in order to design and to implement the solution we propose:

- BPM System: represents the technological solution we have used in order to manage the entire business processe life cycle, including their modeling and execution;
- Domain knowledge base: represents the technological solution we have used to store the domain concepts, along with the relationships between them. In effect, making use of the domain concepts stored in the knowledge base, it is possible to support both the business process modelling, along with the engineering of the UX schema of the prototype;
- IDM: represents the methodology, along with the correspondent notation, we have used in order to model the schema related to the interactions between the user and the application prototype;
- Model Driven approach: represents the reference approach we have used to implement the process, able to engineer the UX schema of the prototype. Such a process is based on a series of models, along with the necessary transformations to automatically generate a target model starting from a source one.

3 Modelling laws and regulations

Both the design of business processes and the modelling of laws and regulations are well-studied fields, even if in an independent way. The problem arises when it is necessary to exchange knowledge between the legal professional and the business process analyst. The first professional figure is expert in the regulatory domain, along with how to interpret the laws and the regulations. The second one has particular competence in how to design information systems, especially with regard to laws and regulations.

In (Guido et al. 2015) the authors propose some helpful guidelines for legal professionals in the definition of a schema that can support the business process analysts during the modelling of processes that must be regulatory framework compliant. Furthermore, "it is important to take into account laws and regulations from the beginning of the design phase. Bearing in mind that laws and regulations could change very quickly, the information system must be flexible in order to evolve rapidly without great effort."

These guidelines comprise five steps:

- Definition of the glossary of terms and important concepts: terms and important concepts are often distributed in the text of the analysed laws and regulations;
- Relationship Identification: the present step aims to identify the relationships between the concepts defined in the previous step;
- Identification of the involved actors;
- Definition of the actions and the groups of actions that an actor must take in order to comply with the laws and the regulations;
- Definition of the temporal relationship between actions to specify the level of priority.

The final result of the above steps may be a valid support in the design of a knowledge base, able to contain all the concepts and relationships directly retrieved from laws and regulations. "Using both the schemes of laws and regulations and the conceptualization in a knowledge base" as the authors assert in the introduction of their paper, "the designer can certainly reduce their learning curve and simply understand complex laws and regulations."

4 Modelling and management of the domain knowledge base

The architecture we propose in this paper is characterized by the interaction with a knowledge base layer related to the health, safety and environmental protection context in workplaces. In this chapter, we report the solution fully discussed in (Paiano et al. 2015) and adopted in order to model and to manage the knowledge of a domain of interest.

Using the analysis of the application domain, the authors of the above quoted paper have reached the following results:

- Entities identification such as: process, worker, regulation, working environment, risk, event, social networking and competence;
- Identification and reusing of some existing ontologies to describe the domain of interest.

As argued in the paper, the knowledge model is divided into two ontological levels:

- *Enterprise Domain Ontologies* (EDO): "they model the entities (i.e., human resources, tasks, processes, objectives, etc.) that represent the organizational environment from a general perspective. Because of their general purpose, these ontologies have been defined extending and combining ontologies (i.e. reference ontologies) already existing at the state of the art";
- *Application Domain Ontologies* (ADO): "they model domain specific concepts and they support the classification of entities represented through the EDO. The regulations represent a typical example of ADO because they are application specific".

Such a knowledge model is able to support, for instance, the identification of one or more business processes to be executed in order to manage unexpected events, or the design of new business processes which are compliant with the regulations and laws related to the particular risk type.

From the architectural point of view, a knowledge middleware allows interaction with the knowledge base, using RESTful web services. These services are essentially based on both text analysis and classification, and make use of a link discovery approach in order to find every concept.

5 System architecture

As mentioned in the introduction, the HSEPGEST project aims to define and to develop an integrated architecture, which is closely connected to the domain knowledge base.

This section contains the technical details related to the design of the Process Management System Architecture. This architecture consists of the following components:

- *Knowledge Explorer* for the detailed description, the reader should refer to Sect. 5.5;
- *Business Process Editor* for the detailed description, the reader should refer to Sect. 5.6;
- *Web Application Editor* for the detailed description, the reader should refer to Sect. 5.8;
- *Workflow Engine* for the detailed description, the reader should refer to Sect. 5.7;
- *User Experience RIA Prototype generator* for the detailed description, the reader should refer to Sect. 5.9.

The architecture is used by business analysts to perform the following activities:

- Business processes modelling: during all the corresponding activities the designer can refer to the concepts present in the knowledge base;
- Process instances execution and monitoring: such activity provides also an interaction with the knowledge base in order to retrieve and to send the processes execution information to the BPM solution;
- Unexpected events management: the unexpected events may be associated to risk types or may come from changes occurring in the enterprise application domain and/or in regulations and laws. In effect, when a change happens in regulations and laws, as a consequence, such change must be propagated in the knowledge base;
- User Experience Modelling of the application prototype: the designer can define some integration points with both the knowledge base and the workflow engine solution;
- Automatic generation of the web application prototype: starting from the previously designed User Experience model, the Process Management System Architecture automatically generates the corresponding web application.

To better understand the components of the architecture, it is expedient to define the elements named sub-process templates and the motivations of their employment. Sub-process templates are blocks of BPMN elements, useful to the business analysts in order to model executable business processes. Unlike business processes, the sub-process templates are not formally correct and, therefore, workflow engines are not able to execute them.

The architecture we propose, in effect, besides allowing the business processes design, also by using regulatory and business information present in the knowledge base, makes possible to directly execute the same processes, in the context of a web application prototype. Such prototype is developed by the designer in order to manage a specific application domain and is able to directly interface with both the knowledge base and the workflow engine. The generation of such an application prototype must be automatically performed and must be appropriately configurable. The designer, in effect, uses the system that we propose to model the UX of the prototype by referring to the knowledge base, in order to define some integration points with information related to laws, regulations, organizations, business processes, etc.

The architecture provides a set of software components that can be arranged in order to support each of the four macro-phases that temporally characterize the process life cycle, and which are named: Planning Time, Design Time, Situation Time and Run Time.

Figures 1 and 2 present, respectively, the modules of the architecture involved during the phases of Planning Time and Design Time and the phases of Run Time and Situation Time.

In the following, we provide a brief description of each phase, aimed to essentially identify the modules of the architecture. A more detailed description, however, is provided in the subsequent sections of this paper.

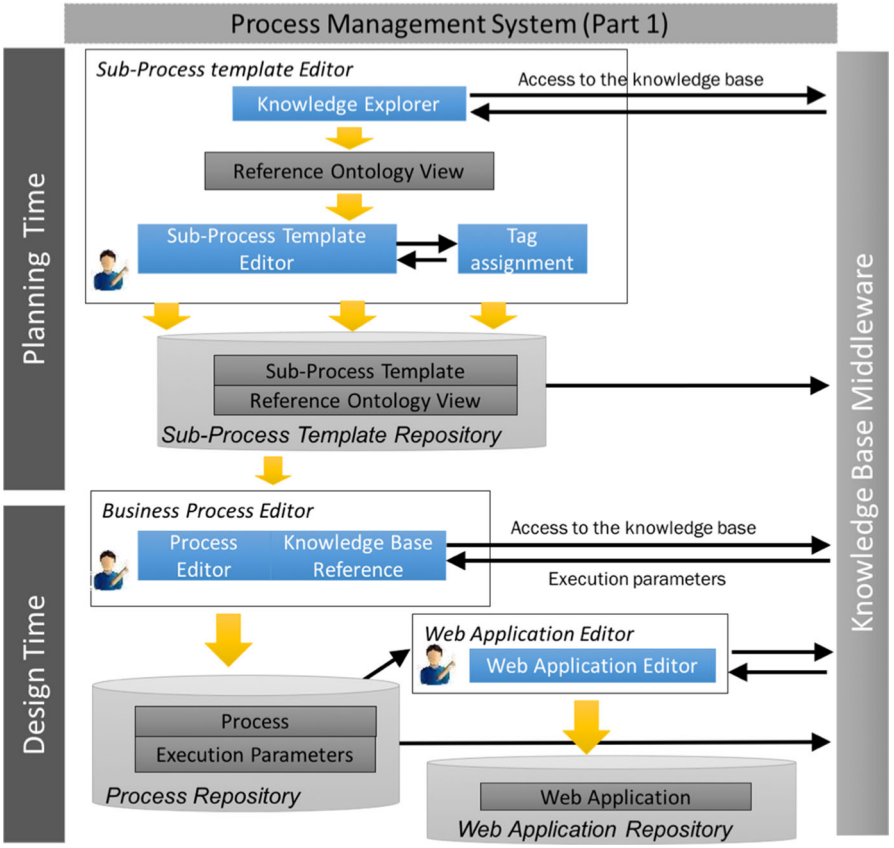

Fig. 1 Process management system architecture (planning time and design time)

5.1 Planning time

In the Planning Time phase, the process designer performs the following activities:

- Selection of knowledge base concepts that characterize the model using the *Knowledge Explorer* component;
- Modelling of sub-process templates using the development environment made available by the design editor.

During the exploration of the knowledge base, in effect, the process designer is completely supported by the editor, which uses the API made available by the KB Middleware module in order to query concepts/resources stored in the knowledge base.

The design of the sub-process templates is performed using the *Business Process Editor* component, able to support the process designer in both the definition of BPMN-compliant models and the semantic characterization of the elements of the

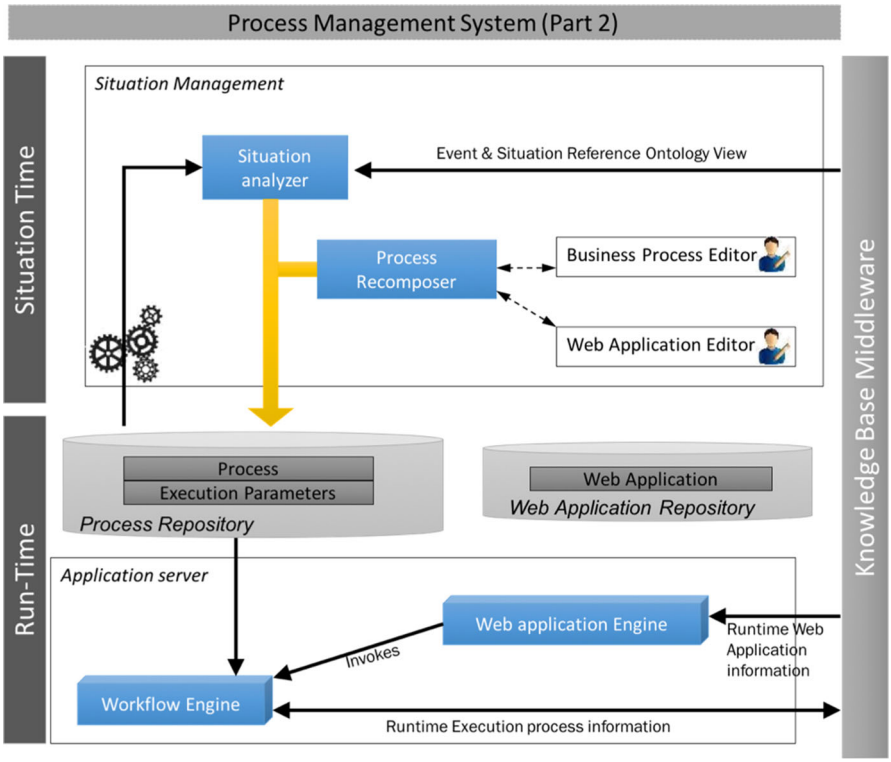

Fig. 2 Process management system architecture (run time and situation time)

processes. In this regard, in effect, the *Business Process Editor* allows mapping the elements of a process to the concepts/resources stored in the knowledge base.

The output generated by the *Business Process Editor* is the model related to the sub-process template, which is represented using a format suitable for transmission to the knowledge base, and compatible with the characteristics required by the workflow engine.

5.2 Design time

In the Design Time phase, the process designer models the process using the *Business Process Editor* component. The definition of the model is performed using the BPMN primitives and/or composing the sub-process templates previously defined. During this phase, the process designer is supported by both the regulatory and the business information, extracted from the knowledge base. For instance he/she can query the knowledge base to know the name of the tasks to insert in the process model. In order to obtain such information, he/she must enter a descriptive text, a legal text or some keywords.

At the end of the model definition of the BPMN process, the designer must make the same process concrete, by specifying the following information:

- Execution parameters: they are inserted by the designer using the *Business Process Editor* and also using values retrieved from the knowledge base;
- Business logic connected to single process task: it can be inserted by both the process designer and the more skilled staff;
- Model of the User Experience related to the web application prototype to be generated: the web prototype user, in effect, uses the same prototype in order to begin the execution of new process instances.

The modelled processes are represented using a format that is suitable for transmission to the knowledge base and compatible with the characteristics required by the workflow engine.

The designer uses the *Web Application Editor* component to model the User Experience of the web application prototype. In particular, this module refers to the IDM and Rich-IDM methodologies for the purposes of the design of the prototype UX. During the modelling phase of the prototype UX, the designer can define some integration points of information, which are not statically written in the application pages, but are dynamically retrieved from the knowledge base. The designers can also provide some access points to the workflow engine, within the model of the prototype UX, using which the user can start a new instance of the business processes defined above.

The final UX model, expressed in XMl Metadata Interchange—XMI—format, represents therefore the main input used by the *Web Application Editor* sub-module named *RIA Prototype Generator*, for the purpose of the automatic generation of the prototype. A more detailed description of the *RIA Prototype Generator* component is provided in Sect. 5.9.

5.3 Run-time

The Run Time phase represents the phase related to the execution of the previous modelled processes. This activity is completely assigned to the application server, which integrates both the *Workflow Engine* and the *Web Application Engine* components.

To better support the Run Time phase, the workflow engine has been extended in order to allow a more accurate monitoring process, but also to allow sending process execution data to the knowledge base. For the aims of the project, such information is essential in order to adequately support process improvement.

During execution of the process instances, the workflow engine performs the business logic written by the process designer at Design Time. This logic contains the code to query the knowledge base, using RESTful web services technology, and is able to retrieve the values of some process execution parameters such as the number of employees or the name of the workers.

5.4 Situation time

The Process Management System that we propose is able to handle expected and unexpected events. In this regard, our solution provides that the knowledge management system must analyse the event, before sending a notification to the Situation Analyser component. Such activity is performed using a publish/subscribe event mechanism. The notification is characterized by a subset of concepts/resources stored in the knowledge base, and is intended to describe the event in detail.

When a notification occurs, the Situation Analyser performs a comparison between the concepts of the event and those referenced by each of the elements that are part of the BPMN processes and/or of the sub-process templates previously modelled. The purpose of this comparison is to identify the processes, which are particularly suitable for management of the expected or unexpected situation.

If the Situation Analyser identifies a model that is perfectly compatible in terms of concepts with the event, then such model is elected as event handler. Otherwise, however, the Situation Analyser tries to identify a combination of processes and/or sub-process templates, which may be a possible solution to the event. The combination of models related to the processes and/or sub-process templates is performed by the Process Recomposer component, with the collaboration of the process designer. In particular, the intervention of the process designer is oriented to verify the new process, but above all to make the new process concrete.

The suggestion about the business processes compatible with the notified event can be retrieved from the knowledge base, along with information about the person who reported the event, the description of the event and the factor that triggered the event itself.

Each of these phases provides a strong interaction with the KB Middleware module. This component is, in effect, responsible for communication between the process management and the knowledge base management systems. Such communication is used to perform the following activities:

- Access to both the legal and the business information, but also to the suggestions, stored in the knowledge base;
- Sending the semantic representations of the modelled processes;
- Sending information about the process instances execution;
- Reception and notification of both expected and unexpected events;
- Reception and notification of changes in the concepts stored in the knowledge base.

In this regard, it is necessary that the access to knowledge base information is performed in an independent way with respect to how such information is structured/stored.

5.5 Knowledge explorer

The *Knowledge Explorer* module allows the knowledge base exploration, along with the collection of the concepts stored in it. In effect, the designer can identify the concepts stored in the knowledge base by performing one of the following search methods:

- By free text, inserting a text extracted from a regulation;
- By keywords;
- By concepts related to the results of a previous search.

The *Knowledge Explorer* module is integrated into the *Business Process Editor* and can be used during the following stages of the modelling process:

- When creating a process or a process template, in which the designer is interested in entering one or more references to the knowledge base concepts;
- During the process design, in which the designer is interested in characterizing a specific BPMN element of the model, using one or more concepts retrieved from the knowledge base;
- During the process concretization, in which the designer is interested in defining input/output parameters connected to the knowledge base concepts.

The knowledge base concepts used in the models can assume different states. Such states are directly retrieved from the knowledge base and can assume the following values:

- *Active* indicates that the concept is still valid and can be used to semantically describe a process or a BPMN element of the process;
- *Deleted* indicates that the concept has been removed from the knowledge base and, thus, should not be used;
- *Replaced* indicates that the concept has been modified in the knowledge base. In this case, all the concepts and all the processes that are involved in the change must be updated in order to ensure compliance with the knowledge base.

When the status of a knowledge base concept changes from active to deleted or to replaced, it is highlighted by the knowledge base using an appropriate event. Changes in state of knowledge base concepts require the support, and often the manual intervention, of the process designer in order to ensure the integrity of any business process. The process integrity management represents one of the main features that the *Business Process Editor* makes available.

5.6 Business process editor

The *Business Process Editor* is a stand-alone application, developed using the Eclipse workbench and based on version 6.0.1 of the jBPM workflow engine.

For the project, the choice of the jBPM suite is the result of scouting and comparison between the main open source solutions in the context of the business process modelling and workflow management. We have already exposed the survey in the related works section.

jBPM provides a full and stable suite, through which it is possible to model and to run processes, in addition to sessions, process instances and tasks monitoring. The jBPM workspace has the characteristic of being web-accessible and it is able to interface with a repository through the Eclipse-based editor. For this reason, jBPM makes available its own Eclipse plugin, which is particularly oriented to developers. The repository allows users to import into the jBPM console the projects created using the Eclipse-based editor.

Moreover, jBPM, being an open-source project, offers an extreme flexibility in terms of work environment customization. In particular, the jBPM source code extension aims to implement the following features:

- *Modelling business process templates* The process designer can perform the basic functions in order to create, read, update and delete business process templates. These functions are extensions of those offered by the jBPM Eclipse-based editor. The main additional contribution that we provide lies in the communication with the domain knowledge base using the knowledge explorer module. The process designer can enter some text or keywords about the laws and the regulations and/or the business organization, and he/she can retrieve useful information for the design. In particular, he/she can do the following:
  - Use the suggestions obtained from the knowledge base to model business process templates;
  - Semantically characterize the design, tagging it with the knowledge base concepts;
  - Create an association between each BPMN process element and the concepts retrieved from the knowledge base;
  - Export the business process templates in the corresponding BPMN ontological representations, including the references to the concepts of the knowledge base that have been associated and used during the modelling phase;
  - Manage the business process template changes due to events notified by the knowledge base.

- *Business process template transformation into executable processes* In order to make a business process executable, the designer has to specify all its execution parameters. By execution parameters, we mean all the parameters needed to run a process. The run-time parameters, indeed, are directly retrieved from the knowledge base and represent the data that a process task uses at run time. The Knowledge Explorer module is responsible for the definition of an input/output parameter that references the knowledge base. Retrieving information from the knowledge base at execution time is achieved by defining a custom task, which

is managed using a specific handler that implements the required business logic for its execution. For this purpose, such business logic uses RESTful web services calls to access the knowledge base. In particular, such methods can be called to obtain all the information closely related to the execution of a specific process instance. The custom task is available within the BPMN palette offered by jBPM, under the Custom Task section. This aspect is discussed under the section on the Workflow Engine module.

- *Process Recomposer* The *Process Recomposer* implements a feature especially useful when it is not possible to identify a business process that is fully compatible with respect to the event that occurs. In effect, when a particular event is notified by the knowledge base, the system proposes to the designer the processes/sub-process templates list related to that event. The designer can select one or more processes or sub-process templates among those proposed by the system, in order to define the resulting process that must handle the event. The output produced by the *Process Recomposer* represents a new process that is automatically obtained by combining all the processes or sub-process templates previously selected by the designer. The automatically generated process is not final, but must be conveniently managed by the process designer.

5.7 Workflow engine

A workflow engine represents a tool suite that allows creating models and automating business processes. A workflow engine, thus, manages the entire business process life cycle, starting from the workflow design to the execution and analysis of the results.

During run-time, the workflow engine executes the business processes in order to perform the actual activities planned within the enterprise. For this reason, it is necessary to assure the correctness of the process, even with respect to all the interactions between every process and the external systems. Only after performing and overcoming the above checks, is it possible to proceed with the deployment and the execution of the business activities (processes and regulations) in the runtime environment.

Process monitoring and performance metrics can be realized using dashboard-like tools that provide, even with graphics, useful and updated information intended for high-level experts. The process just introduced can be fully implemented using a business process management system (BPMS).

A BPMS represents a middleware module that is able to offer features particularly oriented to the resolution of specific tasks in different phases.

As already explained, we have analysed different open source workflow engines, and jBPM resulted in the most suitable solution to be used for the purpose of the project. In particular, with regard to the activation and analysis factors of the business processes, jBPM represents the most complete and convenient solution. In effect, unlike other solutions jBPM provides a complete process administration console, along with full support for business dashboard creation and performance indicator monitoring.

Despite being a complete solution as a business process management and monitoring system, it is necessary for jBPM to provide a communication protocol between the workflow engine persistence layer and the knowledge base, even during the process instances execution. Such requirement is necessary in order to perform the following operations:

- Retrieving from the knowledge base all the data necessary to execute the process;
- Sending to the knowledge base all the data about the instantiated and the running process identifier, the user's identity, the process variables and other useful information. Such data are particularly useful to enrich the knowledge base contents, also with respect to further requests and uses.

All that data also can be used to generate appropriate business logic during the execution of every task provided by a process. In particular, the business logic must:

- Call the REST services to both retrieve and read the information stored in the knowledge base;
- Call the REST services to store data into the knowledge base;
- Generate all the necessary methods to implement the connection to the workflow engine database;
- Define the procedures to query the persistence layer;
- Process the retrieved information;
- Parse the results of the REST calls, in XML format, related to the queries performed on the knowledge base;
- Perform the XML parsing of the data to be sent to the knowledge base using REST invocations.

jBPM provides the ability to manage additional information using a domain-specific task called '*Custom Work Item*'. In particular, in order to define a *Custom Work Item* it is necessary to introduce the corresponding element in a process using the Eclipse-based editor, but also to create a '*Custom Work Item Handler*', which represents the Java class that jBPM calls during the custom task execution of a running process.

The necessity to refer to the *Custom Work Item* stems from the need to:

- Define additional properties related to the execution and the run-time process parameters retrieved from the knowledge base;
- Send information about the parameter values, the custom tasks and the process in execution.

The *Custom Work Item Handler* Java class is used to run (or to stop) the related work items. In particular, for the purpose of the project, we have defined the custom task called TASK_HSPG_CUSTOM, which is managed by a work item handler and whose methods implement the following operations:

- Retrieving information about the running custom tasks, or, in other words, about the entered parameters;
- Performing the necessary business logic to interact via web services with the knowledge base and the jBPM logs (such logic must be partly written by the process designer);
- Informing the process engine about the completion of the work item (or about the abort).

The interaction between the custom tasks and external systems is synchronous, that is the engine is able to intercept that the logic and, therefore, the interaction are finished and can proceed starting the next task in queue.

5.8 Web application editor

The Web Application Editor represents the integrated environment for UX designing and for the correspondent RIA prototype generation. The obtained RIA prototype implements the UX design schema, and can support the execution of the business processes. The necessity to make use of a specific methodology, along with the particular characteristics of the Web Application Editor, requires the adoption of architectural specifications based on:

- Model-Driven Development approach: the whole UX modelling process is based on the definition of models and meta-models, along with their respective transformation;
- IDM methodology: given that IDM represents the reference methodology for the UX modelling process, the editor has to grant full support to the design phases provided by this methodology.

The structure of the User Experience engineering process stems from the architectural choices adopted for the Web Application Editor, and clearly follows the specifications defined by the IDM methodology. In particular, the UX design process is constituted by a series of operations, using which the designer creates the final UX model, and the corresponding RIA prototype, implementing the design steps provided by the IDM methodology.

To support such a process, we introduce a framework, which has an architecture that is able to provide a graphical design environment for the UX modelling, along with a software tool that can generate the correspondent RIA prototype, starting from the final UX model.

The graphical design environment is actually made up of three editors, one for each phase provided by the IDM methodology. These editors are aimed at supporting the designer in the model definition, and in the corresponding diagrams, expressed using specific IDM notations.

The architecture of the framework we propose is therefore made up of the following components:

- IDM Editor: represents a graphical editor for the UX design, obtained by the integration of the C-IDM, L-IDM and Rich-IDM editors;
- RIA prototype generator: represents the software module for the RIA prototype generation, which is derived from the UX model that the designer defines using the IDM editor.

Before moving on to analyse the necessary requirements for each IDM editor component, we briefly summarize the purpose of the diagrams provided by IDM notation:

- C-IDM: represents the point of contact between the conceptual design of a Web/RIA application and the application domain;
- L-IDM: represents a greater characterization of the content introduced in the C-IDM, which is further detailed in order to produce proper User Experience modelling;
- Rich-IDM: represents a review of the UX model defined in the C-IDM, which aims to optimize the user/application interaction, in accordance with the basic principles laid down in a typical RIA application.

During the design of both L-IDM and Rich-IDM diagrams, the designer can interact with the knowledge base in order to retrieve the necessary suggestions about the domain that he/she intends to model.

C-IDM Editor represents a visual development environment for C-IDM diagram editing. The designer uses this editor to start the UX design process of the RIA prototype. In order to obtain a UX pattern of quality, the designer must have a deep knowledge of the enterprise information domain. The outputs produced by the editor are the C-IDM diagram and the corresponding XMI model.

L-IDM Editor represents the visual development environment for L-IDM diagram editing. The editor must be able to automatically produce the L-IDM diagram, through the direct transformation of the C-IDM model. At the completion of the automatic generation, the designer must be able to intervene on the diagram in any case, also in order to further detail the contents to present to the user. To this end, the editor must allow the designer to be able to refer to the concepts and to the relations provided by the legal and business knowledge base. The outputs produced by the editor are the L-IDM diagram and the corresponding XMI model.

Rich-IDM Editor represents the visual development environment for Rich-IDM diagram editing. The editor must be able to automatically produce the Rich-IDM diagram, through the direct transformation of the model expressed in XMI format.

During the Rich-IDM diagram generation, the editor must be able to interface with:

- jBPM repository: in order to retrieve the Business Processes list;
- JBoss application server: the aim is to get the list of the roles and the list of the users that are authorized to access the jBPM web console. Such an instance, in effect, represents the embedded jBoss installation provided by jBPM.

At the completion of the Rich-IDM diagram generation, the editor has to create the XMI file for the corresponding model, also in order to complete the diagram rendering. The designer uses the editor to associate either a context view or a user experience core that belongs to the diagram with:

- A reference to a business process defined in the jBPM repository;
- A role defined in the jBoss application server;
- A user defined in the jBoss application server.

In particular, the association between a Rich-IDM element and a business process enables the designer to create some access points within the UX model, capable of achieving a level of interaction between the RIA prototype and the jBPM workflow engine.

In effect, within the UX model of the RIA prototype, an association between a *context view*, or a *user experience core*, and a jBPM process represents a hyperlink, by which the user starts a new instance of the corresponding jBPM process. Moreover, through this connection the prototype user has the ability to access, in an integrated manner, the jBPM web console. Contextually, the RIA prototype must be able to make available some values of the UX controls to the jBPM web console, as execution parameters of the process itself. For this reason, the IDM editor must allow the designer to identify and to annotate which of the UX control values should be considered start-up parameters for the new instance of the jBPM process.

In contrast, an association between *context view*, or *user experience core*, and a jBoss role and/or user allows limit the visibility of the corresponding UX element to the role and/or the user selected. In effect, for the purposes of UX modelling, the editor must allow the designer to manage access to the contents of a *context view* or a *user experience core*, during the Rich-IDM diagram editing. For this reason, the content provided by a *context view* or a *user experience core* not associated with any role and any user, is public and therefore visible to all authorized users of the RIA prototype.

In order to structure the information content provided by the UX prototype, the editor must allow the designer to refer to the concepts and relationships stored in the knowledge base. In effect, during the Rich-IDM diagram editing, the designer can further detail the contents to present to the user, also using the concepts and the relationships directly retrieved from the knowledge base. In this way, the editor allows the designer to define some integration points between the information content provided by the UX model, and those recovered from the knowledge base. These integration points are particularly useful when the prototype user is committed to carry on business, but especially in the presence of an access point to jBPM. In the latter case, the designer can model the prototype UX trying to substantiate as much as possible the access points to jBPM, through the addition of contents that are able to support the prototype user during the business process management. Such contents may also be general-purpose, referring to laws and regulations, internal and external to the enterprise, related to the process itself. In this case, the editor allows the designer to define within the UX model some access

points to the jBPM web console, which represent also integration points for the user of the RIA prototype.

In effect, the goal is to create the UX model so that the designer can contextualize any reference to a jBPM business process, in an attempt to characterize the corresponding access point. This goal is much more productive as the designer can organically structure the normative and informative reference for the process. In this case, the prototype user is supported in a conscious and thorough manner during the course of his/her work, and especially during the business process management.

The outputs produced by the editor are the Rich-IDM diagram and the corresponding XMI model.

5.9 User experience RIA prototype generator

The RIA Prototype Generator needs to have the availability of the Rich-IDM model of the UX the designer has previously produced using the IDM editor.

The generator must be able to implement a prototype capable of satisfying the following requirements:

- Is a Java servlet-based application;
- Is Java Enterprise Edition-compliant;
- Adopts the Model-View-Controller—MVC—pattern;
- Is a Rich Internet Application;
- Must be able to interface with the knowledge base in order to retrieve the concepts and the relationships introduced by the designer in the UX model;
- Must be able to interface with the workflow engine reference, in order to allow the user to start a new instance of a business process. In this circumstance, the prototype must identify the UX controls, whose values have to be used as execution parameters of the process;
- Must be able to retrieve the list of tasks that are assigned to the user logged on the prototype.

Considering that the prototype must adopt the MVC pattern, the generator must be able to implement both the *View* and the *Model* components of the RIA application.

In particular, to implement the *Model* component of the prototype, the generator must make use of *POJO—Plain Old Java Object*—classes, which essentially have to be JavaBean-compliant. The *View* component makes use of interfaces developed through the *FreeMarker* template engine, primarily for compliance with the reference technology adopted by the jBPM workflow engine.

At the completion of the generation of the *View* and the *Model* components, the generator has to deploy the prototype on a web container. In effect, before starting the prototyping process, the designer must be able to specify the local path to the installation directory of the reference servlet container, in our case Apache Tomcat.

At the completion of the deployment, the generator must also start the Tomcat instance and open a browser window at the HTTP address of the prototype homepage.

At this point, the designer can control the prototype UX, and possibly take action on the graphic aspects of the prototype, using a *WYSIWYG HTML* editor.

In order to work properly, the RIA prototype must also provide the following services:

- Identification of the UX contents to recover from the knowledge base;
- Periodic reload of the contents retrieved from the knowledge base;
- Recovery and periodic reload of the tasks list from the jBPM web console assigned to the user and/or the profile of the user logged on the prototype;
- Access management to the contents offered by the prototype, according to the username and/or the competence profile of the user logged on the prototype itself;
- Integration of the jBPM web console window within the prototype graphical interface;
- jBPM process instance start management.

6 The user experience design of an information system: the business process integration

Figure 3 shows the UX modelling process of the RIA prototype that we propose. This process is characterized by three design phases, called Design Time, Prototyping and Run Time. As noted, Design Time and Run Time are characterized by a strong interaction with both the knowledge base and the business processes, during the whole UX modelling process.

The Design Time is essentially based on the definition of the UX schema of the prototype, making reference to a Model-Driven approach, applied to the IDM methodology. In particular, the Design Time phase involves the use of three editors, capable of realizing an integrated design environment, through which the designer

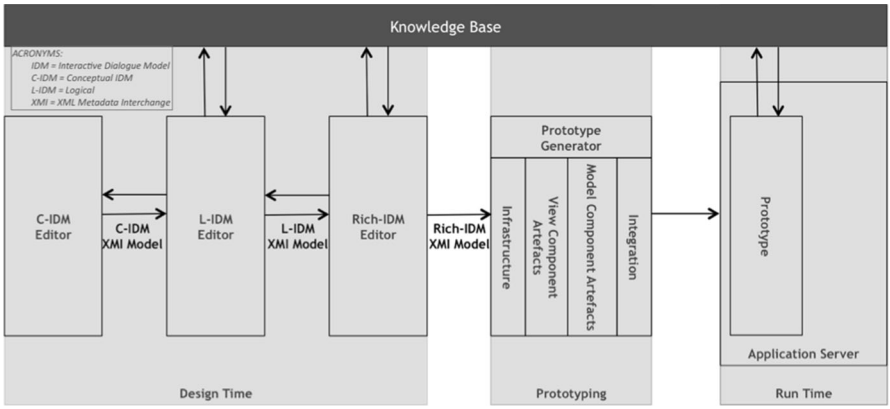

Fig. 3 The UX modelling process of the RIA prototype

can model the UX schema of the prototype, applying the principles of the IDM methodology.

Furthermore, according to the Model-Driven approach principles, the realization of such a design environment is bound to the definition of a meta-model for each editor to implement. These meta-models, in effect, characterize the set of elements, and their attributes, which are available to the designer during the generation of the corresponding model. Each editor, therefore, produces as output a model, in XMI format, which is used as input by the component that follows in the UX modelling process. The model that a component receives as input, in effect, is used by the component itself to automatically produce a first version of the diagram. This diagram is then made available to the designer, so that he/she can make the appropriate changes. The final result of the Design Time phase is the Rich-IDM model of the UX.

This final result represents also the main input used by the Prototyping phase, during which the generator produces the RIA prototype, using precisely the Rich-IDM model of the UX. In particular, the prototype generation is performed through the following phases:

- Infrastructure;
- View Component Artefacts;
- Model Component Artefacts;
- Integration.

The prototype generation ends with the deployment on a web container of the prototype itself. In effect, this operation starts the Run Time phase of the UX modelling process, during which the prototype is complete and working.

After introducing the design phases that characterize the UX modelling process, the following represents the detail of each activity planned, trying to particularly highlight aspects related to the integration of the prototype with the knowledge base and the BPM solution.

1. The designer uses the C-IDM editor to model the corresponding diagram, in order to represent concepts, and their relationships, which identify the specific business domain. In particular, the designer defines the macro-arguments to be made available to the user of the prototype, in the form of *topic*, *relevant relation* and *group of topics*. Given the high level of detail that characterizes the C-IDM diagram, there are no integration points either with the knowledge base or the BPM solution;
2. The C-IDM model generated in the previous step represents the input used by the L-IDM editor for the automatic generation of the corresponding diagram. At the completion of the diagram editing, the designer can still intervene on both the design and the contents provided by the L-IDM diagram, in order to align it with his/her design ideas. The designer, in effect, may decide to further detail the macro-subjects defined in the previous step, referring also to the concepts, and to the related relationships, present in the legal and business knowledge base. To this end, the designer uses the component called *Knowledge Explorer*,

which is available in the IDM editor, to define the first integration points in the UX model of the prototype. The *Knowledge Explorer* component represents the same module used and described in the *Business Process Editor module* and it allows communication with the knowledge base, using a knowledge base middleware and some RESTful web services. The designer uses the L-IDM editor also to perform the L-IDM-Rich-IDM mapping operation, which represents the configuration phase of the Rich-IDM diagram generation. During this phase, the designer defines how many and which *Context Views* to create, but especially how to distribute among these the topics of the L-IDM diagram. At the completion of the L-IDM-Rich-IDM mapping phase, the designer may require the L-IDM editor to generate the Rich-IDM diagram, also specifying the access coordinates to the repository of the jBPM business process workflow engine and the home directory of the JBoss embedded installation;

3. Using the configuration specified by the designer, the editor performs the Rich-IDM diagram generation. During this process, the editor provides the following information:

   - The list of the Business Process references, directly retrieved from the repository of the jBPM workflow engine;
   - The complete lists of the roles and of the users authorized to access the jBPM web console.

   In that way, the editor owns what is necessary to ensure the designer to be able to:

   - Create some access points to the BPM solution within the UX of the prototype;
   - Manage the access to the contents provided by *context views* and *user experience cores*;

4. At the completion of the automatic generation, the editor displays the Rich-IDM diagram so that the designer can make the appropriate changes. Similar to the L-IDM diagram, the designer can change the diagram structure, also to increase the level of detail related to the contents to be presented to the user. In particular, the designer has the ability to create, within the UX model, integration points with the knowledge base and access points to the jBPM workflow engine. The designer, in effect, uses an association between a reference to a jBPM Business Process and a *context view* or a *user experience core* that belongs to the UX schema, to implement such access points. The Design Time phase ends when the designer completes the Rich-IDM model editing;
5. At the completion of the editing phase of the Rich-IDM diagram, the designer can require the editor to perform the RIA Prototype Generation. The editor actually delegates this operation to the RIA prototype generator, which makes available the UX Rich-IDM model. The prototype generation coincides with the Prototyping phase, and consists of the following four sub-phases: Infrastructure,

View Component Artefacts, Model Component Artefacts and Integration. In the following, we analyse in particular the purpose of each of these sub-phases:

a. Infrastructure: represents the set of operations that is able to generate the infrastructure of the prototype. Such an infrastructure is not generated from scratch by the generator, but using a web application template that already provides some basic features, like login, logout and session timeout management. In addition, the web application template also implements the required logic for the purpose of integration with both the knowledge base and the jBPM workflow engine. Compared to the knowledge base, the web application template has the ability to recover and to integrate, into the UX schema of the prototype, all the concepts the designer has provided during the Design Time; while, with respect to the jBPM workflow engine, the web application template provides the business logic necessary to interact with the jBPM web console. This interaction is based on the definition of a specific session with the jBPM console, in order to allow the definition of a new process instance, but also to recover the tasks list assigned to the user logged on the prototype.
b. View Component Artefacts: represents the generation phase of the artefacts related to the *View* component of the prototype. These artefacts are directly added to the previously generated prototype infrastructure, through the processing of each element that makes up the UX Rich-IDM model. Moreover, in this phase, all the access points to jBPM provided by the designer are properly prepared in the prototype UX, in the form of hyperlinks.
c. Model Component Artefacts: represents the generation phase of the artefacts related to the *Model* component. These artefacts are Java implemented and compiled directly through the processing of particular elements of the Rich-IDM model. These elements are called Slot, and although not part of the Rich-IDM notation, are exclusively used for detailing the contents provided by the UX.
d. Integration: represents the recovery phase of the contents residing in the knowledge base and which are provided by the UX of the prototype. The result of the Integration phase is the working RIA prototype, which, as such, may be published through the operation of deployment in a web container.

6. The deployment operation makes available the prototype to the designer, who thus can directly check the result of the UX model designed in the form of a web application. Following this verification, if the designer needs to exclusively act on the objects arrangement of the prototype graphical interface, all he/she has to do is to directly edit artefacts of the *View* component, by using a *WYSIWYG HTML* editor. On the contrary, if the designer finds that the prototype is not entirely suitable with his/her own design ideas, he/she may decide to return, even several times, to the Design Time phase, in order to better

align the UX model. At the completion of the editing phase of the Rich-IDM model, the designer can proceed again with the prototype generation, and with the usual verification of the prototype itself. This verification primarily aims to examine the prototype performance, from the end user perspective, with respect to both the integration points and the jBPM access points. The prototype, in effect, is capable of interacting with both the knowledge base and the BPM solution. Compared to the knowledge base, the prototype implements a specific service, able to periodically retrieve the concepts to be presented in the UX, in order to keep them synchronized with respect to the knowledge base itself. The integration with the workflow engine, however, is triggered by the user, through the use of appropriate links on the UX, and then completed by the business logic implemented by the prototype. This logic, in effect, first and foremost deals with starting a new jBPM process instance, but also to recover from the prototype UX all the control values to be used as start-up parameters for the process itself.

7 Case study

To further explain the structure of the architecture that we propose and how it works, in the following we analyse a case study concerning a process of health monitoring.

Health monitoring is part of the HSEP processes, which target the periodic check of the health of the enterprise workforce. The critical situation of this process is associated to the factors:

- Danger and risk related to the activities performed by the enterprise staff;
- Health of the environment in which these activities are carried out;
- Contact with substances dangerous to human health;
- Existence of transformation processes that can affect the health of employees.

The high critical factor linked to the process of health monitoring, as well as the constant developments in the regulatory and legislative environment, makes this process particularly interesting for the purposes of this case study. The regulations related to health monitoring, in effect, undergo improvements and integrations that need to be interpreted and adopted in a short time.

Next we indicate the expected aspects for the description of the health monitoring case study:

- Modelling of business processes and sub-process templates, using BPMN and BPMN 2.0 specification;
- Knowledge base concepts association with the business process and/or with BPMN elements;
- Saving the process design in a format compatible with the workflow engine, but also in an ontological format that can be uploaded into the knowledge base;

- Identification of the topics and definition of the correspondent C-IDM diagram of the UX prototype;
- Generation and editing of the L-IDM diagram;
- Generation and editing of the Rich-IDM diagram;
- Generation of the RIA prototype related to the scheme of the designed UX;
- Start of a new instance of the process of health monitoring.

We want to underline that in the steps listed above, the information extracted from the domain knowledge base gives considerable support, especially during the business process modelling phases.

We analyse in detail each step.

Figure 4 shows the process schema related to the health monitoring. The notation used is BPMN. The actors involved in the process of health monitoring are: the employee, the employer and the doctor in charge of safety at work. The health monitoring is periodically carried out by the doctor in charge, when requested by the employer or by the employee himself. In the case study we propose, we take into account the activities carried out by the doctor that are related to the medical examination and to the subsequent result communication to both the worker and the employer.

The planned activities for the doctor are:

- He identifies the employee;
- He acquires the documentation;
- He checks the documentation presented by the employee, and if it is in paper format, he will copy it on the electronic medical record of the worker;
- He performs the medical examination;
- He subjects the worker to the examination;
- He generates the diagnosis based on both the documentation presented by the worker and the health condition observed during the examination;

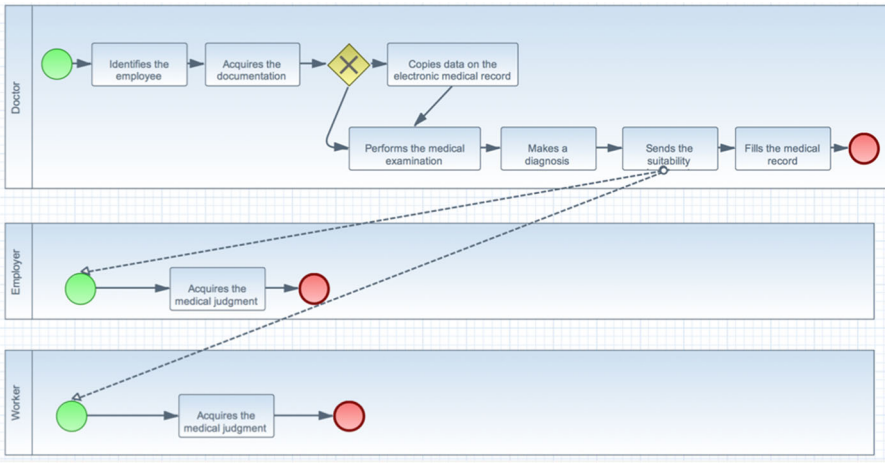

Fig. 4 BPMN design of the sub-process 'Visit performing'

- He sends the suitability judgment to both the employee and the employer;
- He updates the medical record of the worker.

The activities provided for the employee and the employer, in the case study, are only limited to the consultation of the results of the medical examination.

The process design, described and shown above, is obtained using the proposed architecture (in relation to the phases of planning time, design time, situation time and run time) and the already defined tools.

During the first phase, we use the wizard to create a new process or sub-process template and the knowledge explorer tool, in order to extract concepts from the knowledge base and to associate them to the model. Figure 5 shows the tool used for the knowledge base exploration. Such tool is generally called during the creation of a new business process or sub-process template.

Obviously, the selected concepts are directly linked to the respective normative references in the knowledge base.

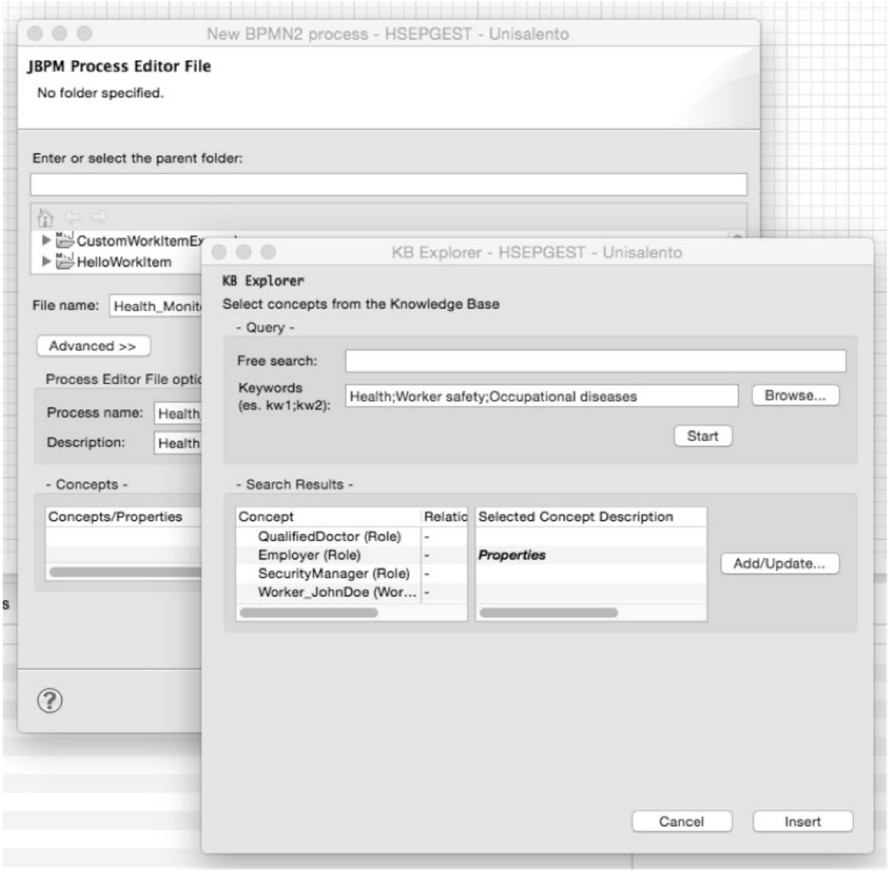

Fig. 5 New process wizard and knowledge explorer

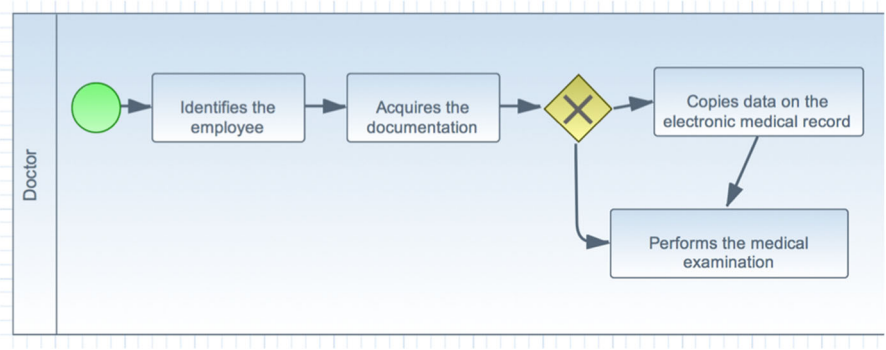

Fig. 6 Sub process template 1

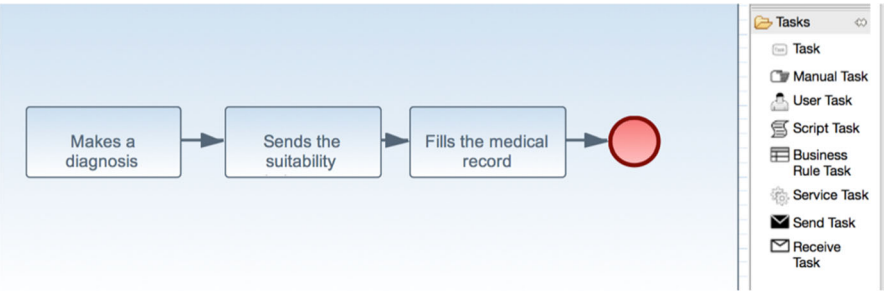

Fig. 7 Sub process template 2

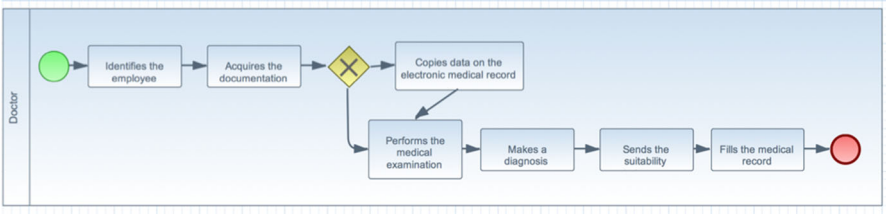

Fig. 8 Sub process template composition

We create distinct sub-process templates to be composed in a single process; we use the wizard to create a new sub-process template. In this context, the features we use are exactly the same as those provided in the wizard to create a new process, except for the validation check that, in the case of the template, is not carried out: this is because templates could be formal incorrect. The two sub-process templates and their final composition are respectively displayed in Figs. 6 and 7, and in Fig. 8.

The composition is automatically performed by the system, based on the template input by the designer, without the sequence flows that connect the elements of the different templates. The last aspect has to be manually handled by the designer.

Therefore the process design can be realized using previously modelled process templates or the BPMN primitives. During this phase, the designer can query the knowledge base in order to both obtain support and associate knowledge base concepts to each element of the BPMN notation.

At this point, through the knowledge explorer, it is possible to insert into the process design relevant information for the execution: users covering the role described in the process, specific input for each task, etc.

The concretized process is saved in both a semantic format and a format that is useful for the process execution within the workflow engine.

In addition, the necessary code is written to implement the business logic of the used custom tasks.

In parallel, the designer can start the prototyping generation process with the User Experience schema definition. In effect, using the web application prototype, the prototype user can execute the health surveillance process in a consistent way with both the workflow engine and the information retrieved from the knowledge base.

Figure 9 shows the C-IDM diagram of the prototype UX. For the purposes of the UX schema generation, the designer identifies the following topics:

- Firm: represents the Single Topic related to the information content that aims to present the enterprise;
- Firm News: represents the Single Topic relating to an additional information content, which describes in detail the mission and the enterprise itself;
- Recommendation: represents the Multiple Topic relating to an operation mode provided by a regulation or by a legislation, that must be undertaken for the proper conduct of a particular activity. The Topic also provides a Group of Topics related to the list of possible recommendations;
- Worker: represents the Multiple Topic relative to a generic enterprise employee, and, as such, must periodically undergo health surveillance. The Worker Topic

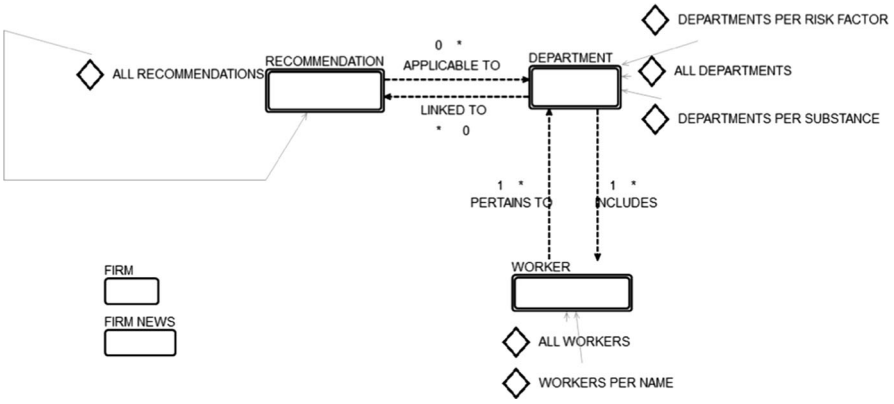

Fig. 9 C-IDM editor—C-IDM design

also includes two Group of Topics respectively dealing with the complete list and the list grouped by the name of the enterprise workers;

- Department: represents the Multiple Topic related to one of the departments that make up the enterprise. The Department binds both the Recommendation and the Worker topics, using specific Relevant Relations. The relationship with the Department seeks to track which recommendations should be taken into account with respect to the activities that are carried out in a department. The relationship between the Department and the Worker keeps track of the staff workers that belong to any enterprise department. The Topic also provides three Groups of Topics respectively dealing with the complete list, the list grouped by treated substance and the list grouped by risk factor of an enterprise department.

The designer directly retrieves all the concepts introduced in the case study by consulting the knowledge base. During the design phase of the prototype UX modelling process, in effect, the designer can use the IDM editor to consult, and then to retrieve, concepts and relationships stored in the knowledge base. In this way, the designer is fully supported during the design phases of the prototype user experience, in order to obtain a result that is the most consistent and the most exhaustive as possible.

At the completion of the editing phase of the C-IDM model, using the corresponding editor, the designer can proceed with the L-IDM diagram derivation, which is shown in Fig. 10. Actually, the designer completes the automatically generated diagram by adding some *Content Dialogue Act* to L-IDM topics. In particular, the *Content Dialogue Act* provided for each topic is presented in Table 2:

During the editing phase of the L-IDM diagram, the designer may better detail the UX contents using the concepts present on the knowledge base.

Before starting with the automatic derivation of the Rich-IDM diagram, the designer needs to proceed with the L-IDM-Rich-IDM mapping operation. This mapping allows the designer to specify which and how many *Context Views* must be

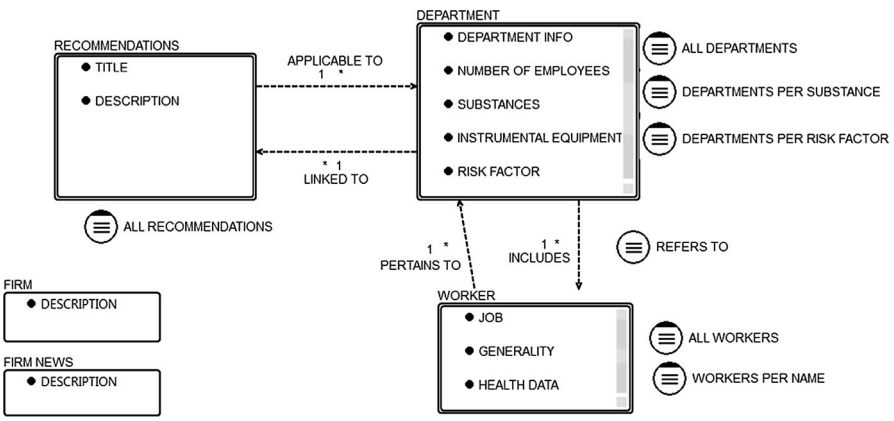

Fig. 10 L-IDM editor—L-IDM design

Table 2 Content dialogue acts list

Topic	Content dialogue act
Firm	Description
Firm news	Description
Recommendation	Title
	Description
Worker	Job
	Generality
	Health data
Department	Department info
	Number of employees
	Substances
	Instrumental equipment
	Risk factor

created, and especially how to distribute among these *Context Views* the topics that belong to the L-IDM model. To this end, the designer uses the corresponding functionality offered by the editor.

The moment the designer assigns a *Topic* to a *Context View*, the L-IDM editor appropriately colours *Topic* itself. The *Topics* assigned to the same *Context View* are coloured using the same shade.

In our case, as shown in Fig. 11, the designer decides to create four *Context Views*: one for Recommendation, Worker and Department *Topics*, while the fourth contains both Firm and Firm News *Single Topics*.

At the completion of the L-IDM-Rich-IDM mapping phase, the designer may decide to proceed with the automatic Rich-IDM diagram derivation, starting from the L-IDM model along with the mapping instructions themselves. For integrating the Rich-IDM editor and jBPM workflow engine, the generation diagram wizard

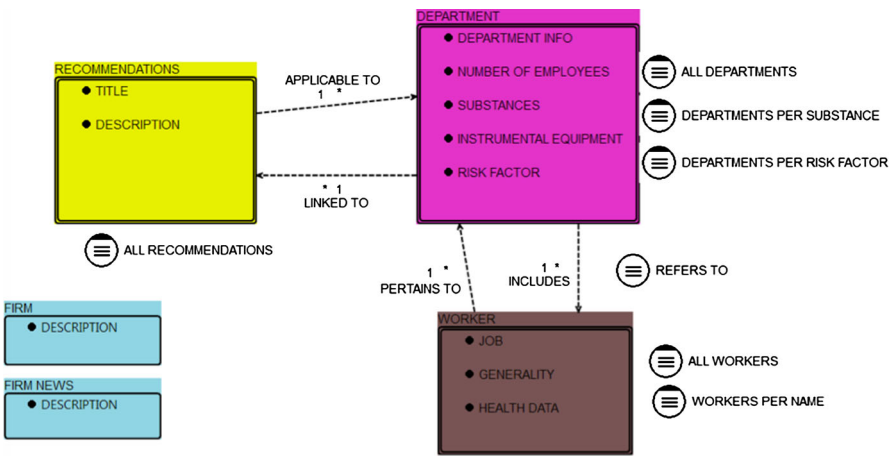

Fig. 11 L-IDM editor—L-IDM-rich-IDM mapping

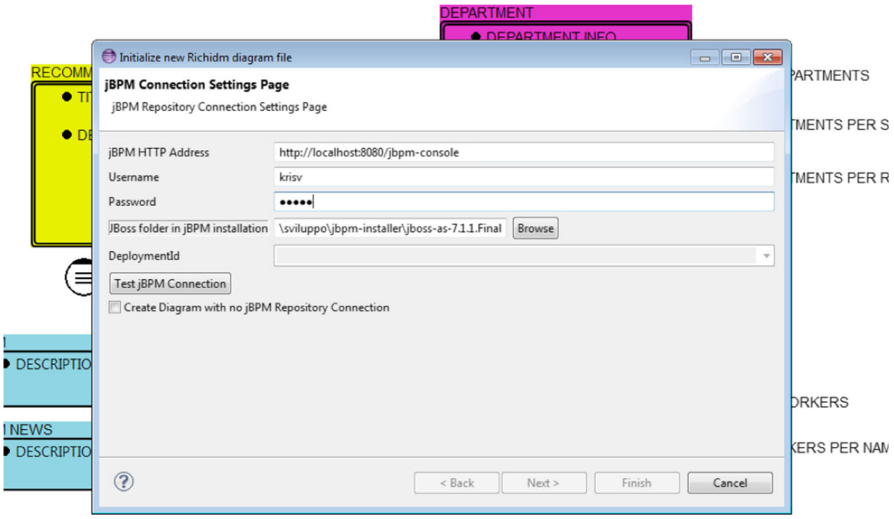

Fig. 12 L-IDM editor—Rich-IDM diagram generation wizard

needs the designer to enter the URL and the credentials to access the jBPM web console, along with the home directory of the jBoss application server installation, embedded in the same jBPM. This information is necessary in order to retrieve the list of the users and the list of the roles authorized to access the jBPM web console. In this way, the designer can manage access to the prototype UX information, by specifying which of the users and/or roles can view which of the *Context Views* and/or *User Experience Cores* of the diagram. Another important item for the purpose of the Rich-IDM diagram generation is the *deploymentId*, whose list is automatically retrieved from the wizard, once the designer has specified the jBPM access coordinates. In effect, in order to access the repository, the jBPM API needs the *deploymentId* information to interrogate the web console, using REST calls. Figure 12 shows the dialog box related to the Rich-IDM diagram generation wizard.

At the wizard completion, the IDM editor shows to the designer the Rich-IDM diagram, corresponding to the L-IDM model. At this point, the designer can intervene on the diagram layout, but also on the structure of the same. Figure 13 shows the Rich-IDM diagram resulting after the interventions of the designer. In particular, the designer has introduced the following Transition RIA-Page Element:

- *Department Worker List* represents the Transition RIA-Page Element that allows the user to start the transition between the Department and the Worker *Context Views*;
- *Recommendations used by the department* represents the Transition RIA-Page Element that allows the user to start the transition between the Department and the Recommendation *Context Views*;

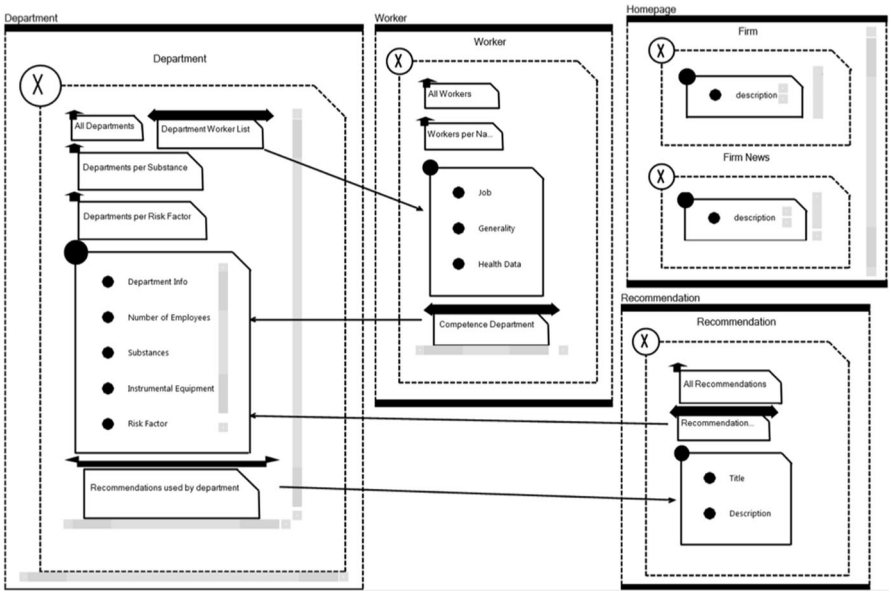

Fig. 13 Rich-IDM editor—Rich-IDM design

- *Competence Department* represents the Transition RIA-Page Element that allows the user to start the transition between the Worker and the Department *Context Views*;
- *Recommendations adopted by department* is the Transition RIA-Page Element that allows the user to start the transition between the Recommendation and the Department *Context Views*;

Among the operations made available by the IDM editor, the designer can associate a jBPM process to a *Context View* or to a *User Experience Core* of the Rich-IDM diagram. Figure 14 shows the Rich-IDM diagram after the designer has associated a *Context View* with the previously defined Health Surveillance business process. The moment the designer associates a *Context View* or a *User Experience Core* to a jBPM process, the IDM editor gives evidence by colouring the corresponding elements in the diagram.

At the completion of the editing phase of the Rich-IDM diagram, the designer can require the IDM editor to perform the RIA prototype generation, starting from the designed Rich-IDM model. Figure 15 shows the dialog box proposed by the IDM editor, in order to start the prototype generation. The parameters the designer has to provide are:

- The Rich-IDM model file path;
- The home directory path to the web container where to deploy the generated prototype. In this case, the reference solution is the Tomcat servlet container.

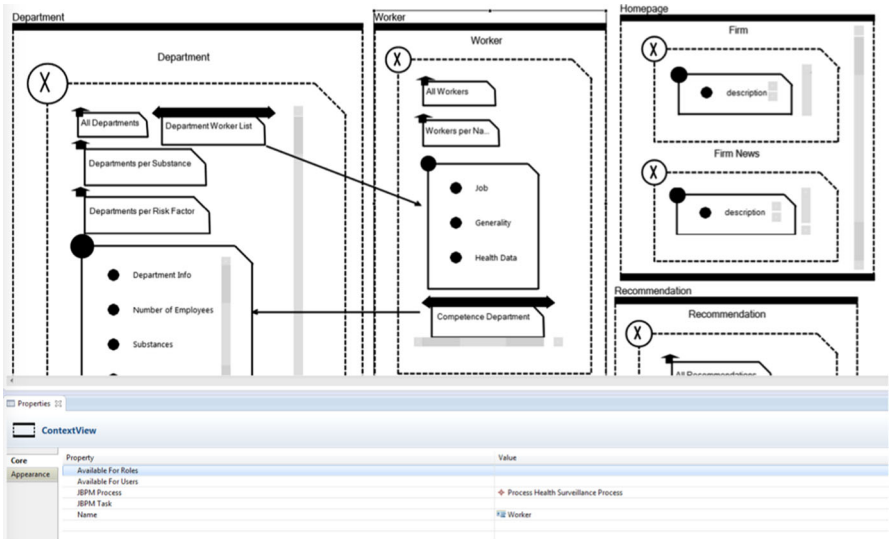

Fig. 14 Rich-IDM editor—context view association with a jBPM Process

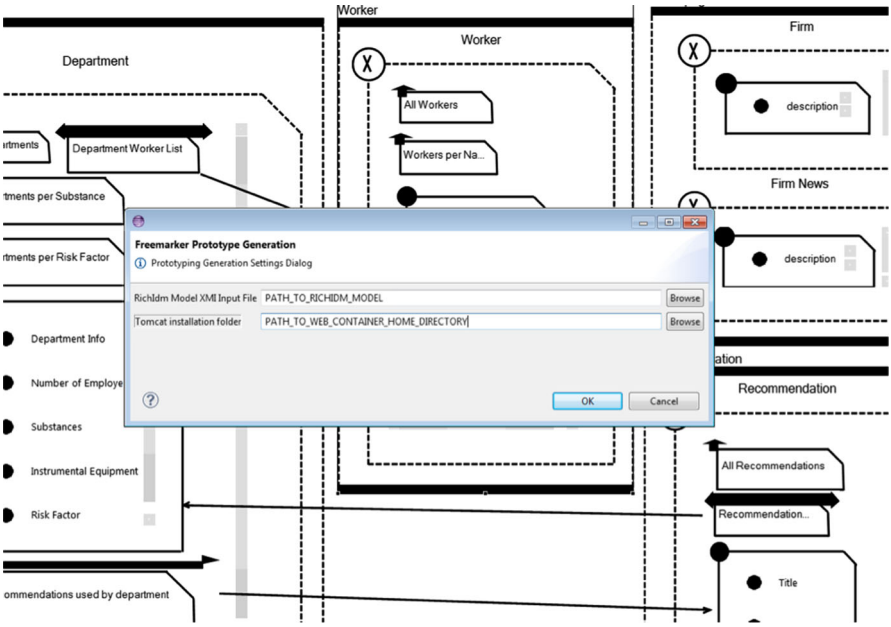

Fig. 15 Rich-IDM editor—RIA prototype generation

At the completion of the generation process, the generator proceeds with the prototype deployment, and then with the opening of a browser window at the prototype login page, as shown in Fig. 16.

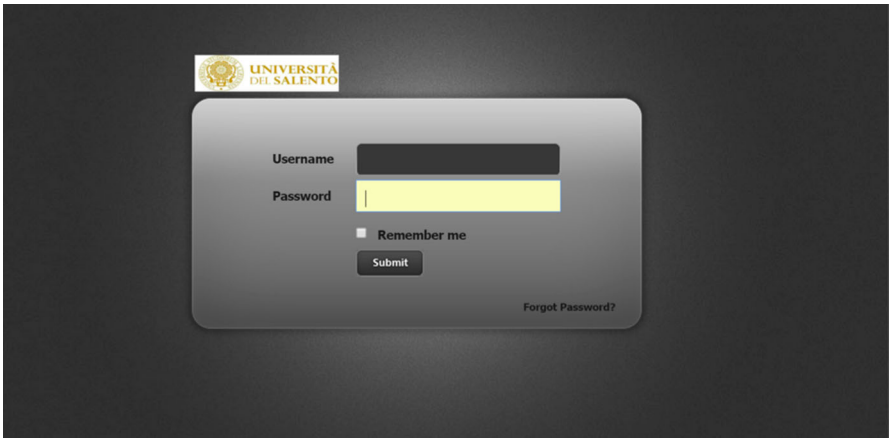

Fig. 16 RIA prototype—login page

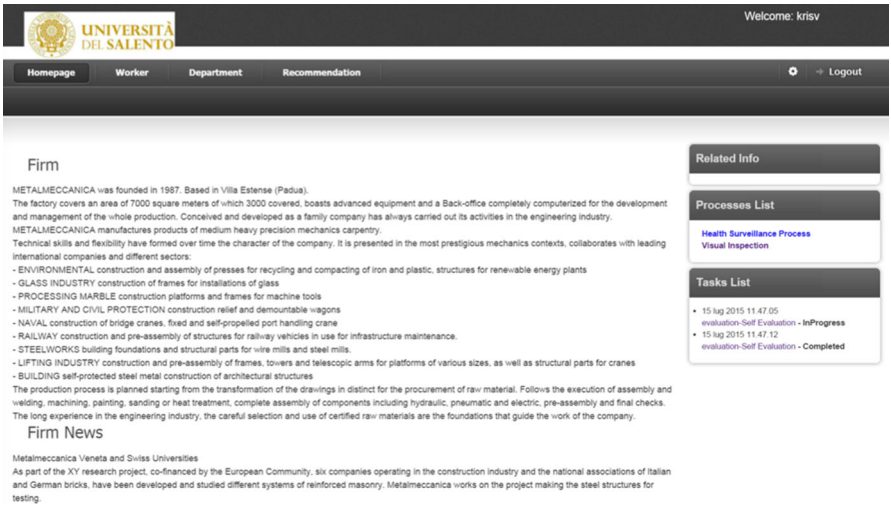

Fig. 17 RIA prototype—homepage

Using one of the user accounts authorized to access the jBPM web console, it is now possible to log in the prototype; if the login is successfully, the prototype displays the homepage to the user, which is shown in the Fig. 17. The figure shows the structure of the prototype graphical interface, which presents on the top a menu bar, while the middle section consists of two columns, one on the left that shows the specific homepage contents and another on the right that includes the subsections: Related Info, Processes List and Tasks List. The menu exactly provides all the references to the *Context Views* previously defined in the Rich-IDM diagram.

By accessing the Worker *Context View*, is possible to note the presence of the jBPM access point corresponding to the previous association between such *Context*

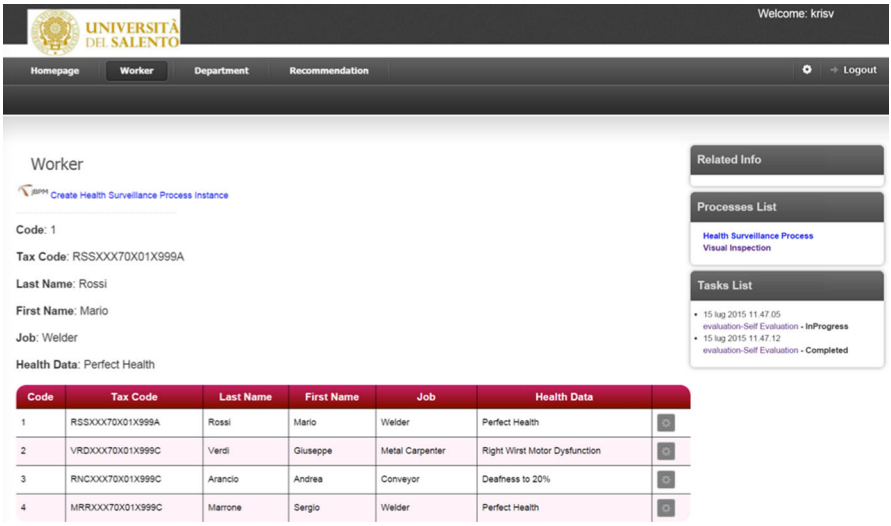

Fig. 18 RIA prototype—examples of integration point and jBPM access point

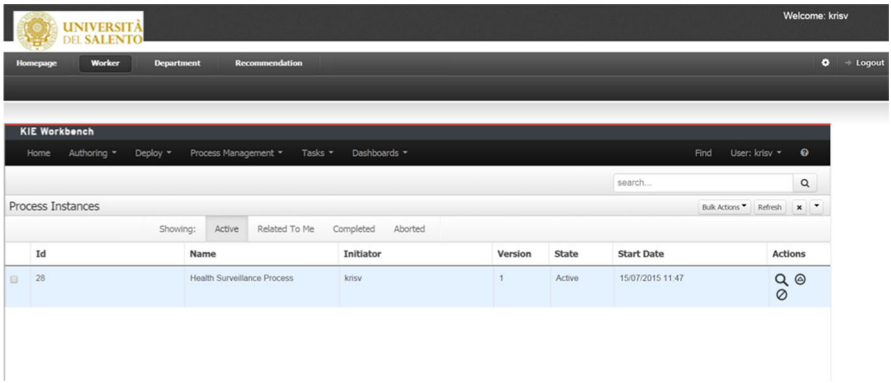

Fig. 19 RIA prototype—jBPM web console integration

View and the Health Surveillance process. The prototype page related to Worker *Context View*, shown in Fig. 18, also reports the enterprise workers list. It is important to note that among the worker generality, the designer has marked the worker's tax code as a start-up parameter for the associated process. Therefore, following the link in the prototype UX in order to start the Health Surveillance process, the tax code related to the selected employee is sent as a parameter to the jBPM web console.

Figure 19 shows the prototype page that appears when the user starts a new instance of the Health Surveillance process. As can be noted, this page fully integrates the jBPM web console window, in which appears the evidence of the process just started.

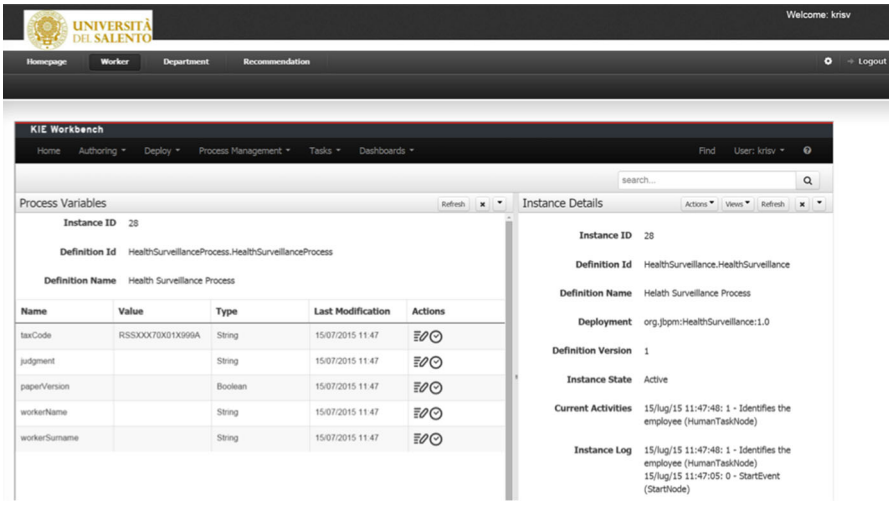

Fig. 20 RIA prototype—jBPM process start-up parameter initialization from the UX prototype

At this point, accessing the process details related to the variables shown in Fig. 20, it is possible to note that the variable value related to the employee tax code coincides with the one previously selected from the prototype page.

8 Conclusions and future work

In this paper we propose an architectural and methodological solution for business processes design and execution, through the extension of the jBPM suite and the prototyping of a web application.

The additional contribution is the strong integration of the domain knowledge base within the architecture. This greatly facilitates designers and business experts during both the process modelling and the critical situation management, in the domain in which they work. The complexity of the knowledge base is hidden by the presence of an intermediate layer between the Process Management System and the knowledge base itself.

In that way, we have completed the future developments exposed within the article (Guido, Paiano and Pandurino 2014), namely the implementation of the overall architecture and the identification of a methodology for the design of process oriented web applications.

As part of future developments, a first goal, to achieve even in the short term, is to reuse the introduced architecture in different application scenarios.

In the longer term, it would be interesting to adapt and expand the methodology for the design of process-oriented applications that we proposed, including the generation of prototypes not necessarily of a web nature.

References

Asif M, Fisscher OAM, De Bruijn EJ, Pagell M (2010) An examination of strategies employed for the integration of management systems. TQM J 22(6):648–669

Beckmerhagen IA, Berg HP, Karapetrovic SV, Willborn WO (2003) Integration of management systems: focus on safety in the nuclear industry. Int J Qual Reliab Manag 20(2):210–228

Bernardo M, Casadesus M, Karapetrovic S, Heras I (2009) How integrated are environmental, quality and other standardized management systems? An empirical study. J Clean Prod 17(8):742–750

Bolchini D, Paolini P (2004) "IDM: interactive dialogue model: toward a dialogue-based approach to design interactive hypermedia applications" deliverable D3—WED (web as dialogue) design framework. http://www.tec-lab.ch/d3_wed_fnrs_105211_1020611.pdf. Accessed 5 Mar 2015

Curcin V, Ghanem M, Wendel P, Guo Y (20070 Heterogeneous workflows in scientific workflow systems. Paper presented at 7th international conference of computational science, Beijing, China, May 27–30

Dumas M, Van Der Aalst WMP, Ter Hofstede AHM (2005) Process-aware information systems: bridging people and software through process technology. Wiley, Hoboken

El Kharbili M, Pulvermueller E (2009) A semantic framework for compliance management in business process management. Paper presented at 2nd international conference of business process and service computing, Leipzig, Germany, March 23–25

Guido AL, Paiano R, Pandurino A (2014) The management of health, safety and environment protection in enterprise processes. Paper presented at 18th world multi-conference on systemics, cybernetics and informatics, Orlando, Florida, USA, July 15–18

Guido AL, Paiano R, Pandurino A (2015) From laws to business process: reducing the skill gap between legal professional and business process analyst. Paper to be presented during the 6th IEEE international conference on internet technologies and applications, Wrexham, North Wales, UK, September 8–11

Kent S (2002) Model driven engineering. In: Butler MJ, Petre L, Sere K (eds) Integrated formal methods. Proceedings of the third international conference on integrated formal methods 2002 Turku, Finland, May 15–18, 2002, vol 2335. Springer, Berlin Heidelberg, pp 286–298

Millet PA, Schmitt P, Botta-Genoulaz V (2009) The SCOR model for the alignment of business processes and information systems. Enterp Inf Syst 3(4):393–407

Monakova G, Leymann F (2013) Workflow ART: a framework for multidimensional workflow analysis. Enterp Inf Syst 7(1):133–166

Mu W, Bénaben F, Pingaud H (2015) A methodology proposal for collaborative business process elaboration using a model-driven approach. Enterp Inf Syst 9(4):349–383

Nostro PD, Orciuoli F, Paolozzi S, Ritrovato P, Toti D (2013) A semantic-based architecture for managing knowledge-intensive organizations: the ARISTOTELE platform. Lect Notes Comput Sci 7652:133–146

Paiano R, Pandurino A, Guido AL, Ritrovato P, D'Apice C, Laria G (2015) An approach to integrated management system exploiting knowledge base to support business processes management. Paper to be published during the 12th conference of the Italian chapter of association for information systems, Rome, Italy, October 9–10

Pandurino A, Bolchini D, Mainetti L, Paiano R (2010) Rich-IDM: extending IDM to model rich internet applications. Paper presented at 12th international conference on information integration and web based applications & services, Paris, France, November 8–10

Pojasek RB (2006) Is your integrated management system really integrated? Environ Qual Manag 16(2):89–97

Sadiq S, Governatori G (2015) Managing regulatory compliance in business processes. In: vom Brocke J, Rosemann M (eds) Handbook on business process management 2. Springer, Berlin, pp 265–288

Seghezzi HD (1997) Business concept redesign. Total Qual Manag 8(2–3):36–43

Tranmer J (1996) Overcoming the problems of integrated management systems. Qual World 22(10):714–718